



CS348 INFORMATION SYSTEMS

Hisham Benotman

Most slides are adopted with permission from D. Maier and L. Delacambre, L. Shapiro, and R. Ramakrishnan

- Concurrency Control
 - Transactions
 - ACID conditions
 - Isolation
 - Schedules
 - Guaranteeing isolation
 - Serializability
 - Locking
 - Two-Phase Locking

Caveat: We'll Ignore Details of the Data

We will see a lot of data items named A, B, etc.

Could be tables, pages, rows

The analyses we consider don't care (much) about granularity.

If you want something concrete, consider them to be rows in a table.

Concurrent processing against the database

- Multiple students reading the same question in Ed STEM.
- Multiple students trying to register in a course.
 - Possibly by updating the attribute: number-of-available-seats for a course.
- Facebook users like a post.
- Multiple transactions accessing the account balance for one customer.
 - Utility company deducting money
 - Loan installment
 - Account debit purchase
 - Salary payment

Hotseat Question

- Which application has more concurrency?
 - Multiple students reading the same question in Campuswire.
 - Multiple students trying to register for a course.
 - Facebook users like a post.
 - Assuming a famous person with a large number of friends/followers
 - Multiple transactions accessing the account balance for one customer (debit purchase, utility billing).

Why Have Concurrent Processing (from multiple users) Against a Database?

- Higher throughput, lower response times
 - Users do not wait in line!
- Better utilization of resources: While one process is doing a disk read, another can be using the CPU or reading another disk.
- But ... concurrent processing can lead to incorrect data in the database or incorrect data returned to users!

Transaction

A transaction is:

- one “complete” set of actions
- defined by the user (meaningful to the application)
 - For example, transfer money (deduct money from account A and add money to account B).

Example Transaction

Transfer \$100 from one account to another:

- read balance in first account
- check balance to see if there's enough money
- read balance in second account
- subtract \$100 from first account balance
- add \$100 to second account balance
- update first account record
- update second account record
- print receipt

Needs to be done “all or nothing”

Transactions (cont.)

User (application developer) must indicate:

- `BEGIN TRANSACTION;`
- `read/write/modify statements` intermixed with other programming language statements

plus either

- `COMMIT;` - indicates successful completion **or**
- `ROLLBACK;` - indicates program wants to roll back (abort the transaction)

Transactions

- For concurrency control purposes (inside DBMS):
 - a **transaction** is one atomic unit of work.
Thus we must be able to undo it if incomplete (e.g., in case of a system failure)
- DBMS views a transaction as (only) a sequence of **reads, writes plus commit & abort**
(mostly ignores the actual SQL that generated the reads and writes)

The ACID Properties of Transactions

- **Atomicity:** A transaction happens in its entirety or not at all.
- **Consistency:** If the DB starts in a consistent state, (this notion is defined by the user; some of it may be enforced by integrity constraints) and a transaction executes with no other queries active, then the DB ends up in a consistent state.
- **Isolation:** The transaction is isolated from other transactions. The effect on the DB is (as if) the transaction executed by itself. Other transactions don't see partial updates.
- **Durability:** If a transaction commits, its changes to the database state persist.

Example: Isolation

- Consider two transactions:

```
T1:  BEGIN  A=A+100,  B=B-100  END
```

```
T2:  BEGIN  A=1.06*A,  B=1.06*B  END
```

- Intuitively, the first transaction is **transferring \$100** from B's account to A's account. The second is **crediting** both accounts with a **6%** interest payment.

Which Transactions are Isolated?

Initial values → for A, B	<u>T2 then T1</u>		<u>T1 then T2</u>		<u>T1, T2 mixed</u>	
	A	B	A	B	A	B
	1000	500	1000	500	1000	500
	1060		1100		1100	
		530		400	1166	530
	1160	430	1166	424		430

These are isolated

Illegal values!
(**NOT** Isolated)

T1:	BEGIN	A=A+100,	B=B-100	END
T2:	BEGIN	A=1.06*A,	B=1.06*B	END

Consistency

- Consistency is defined by the application
- For example,
 - Total accounts balance before money transfer is equal to the total balance after transfer.
 - No isolation -> no consistency

Consistency

- Consistency is defined by the application
 - For example,
 - Number of available seats for a course is between 0 and 200.
 - Begin transaction
 - Read available seats A
 - IF $A > 0$
 - register student
 - Update Set $A = A - 1$
 - Commit
 - Unisolated transactions for course registration may lead to a negative value! (e.g., three transactions read $A=1$ and then decrease A [three times] to -2).
 - No isolation -> no consistency
-

Consistency

- Consistency is defined by the application
- For example,
 - Number of registered courses match tuition fees.
 - Insert registered courses for a student
Insert tuition fees for a student
 - What happens if system crashes between the two insert statements
 - No atomicity -> no consistency (courses are registered but fees are not recorded)

Supporting the ACID Properties of Transactions

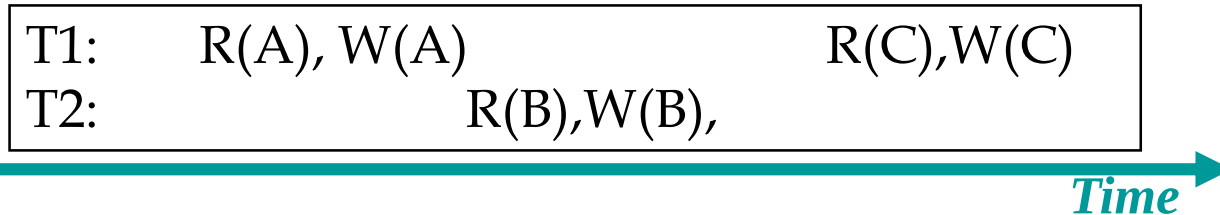
- Recovery System* • **Atomicity:** A transaction happens in its entirety or not at all.
- Programmers* • **Consistency:** If the DB starts in a consistent state, (this notion is defined by the user; some of it may be enforced by integrity constraints) and a transaction executes with no other queries active, then the DB ends up in a consistent state.
- Concurrency Control System* • **Isolation:** The transaction is isolated from other transactions. The effect on the DB is (as if) the transaction executed by itself. Other transactions don't see partial updates.
- Recovery System* • **Durability:** If a transaction commits, its changes to the database state persist.

Concurrency Control

- Schedule
- Serial Schedule
- Serializable schedule

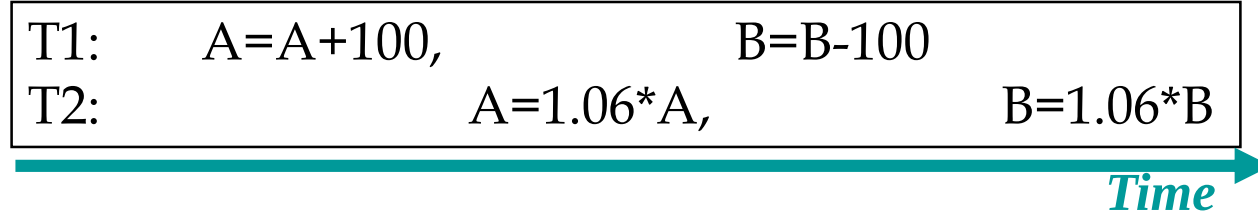
Schedules

- Schedule: An interleaving of actions from a set of transactions, where the actions of each individual transaction are in the original order.
 - Represents an actual sequence of database actions.
 - Example: $R_1(A)$, $W_1(A)$, $R_2(B)$, $W_2(B)$, $R_1(C)$, $W_1(C)$
 - In a *complete* schedule, each transaction ends in *commit* or *abort*.
- Initial State of DB + Schedule $\rightarrow$ Final State of DB



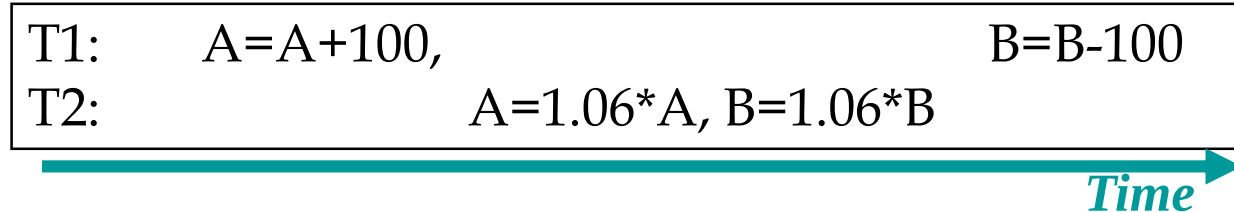
Interleaved Schedules:

Consider this interleaved *schedule*:



equivalent to **T1 then T2** on the prior example. **This is OK.**

Consider another interleaved *schedule*:



equivalent to **third schedule** on the prior example. **This is illegal.**

Serial Schedule

- **Serial** schedules:
 - Run transactions one at a time, in a series. (Different orders might give different results.)
 - Does not interleave actions from different transactions.
 - No concurrency!
- Example Serial schedules:

T1:	R(A), W(A), R(C), W(C)
T2:	R(B), W(B)

T1:	R(A), W(A), R(C), W(C)
T2:	R(B), W(B)

Serializable Schedule $\Leftrightarrow$ Isolated Transactions

- **Serializable** schedules:
 - Interleaves actions from multiple transactions (real concurrency!)
 - **Final state must be the same as the state produced by one of the serial** schedules.
 - The transactions should not have conflicting operations that lead to inconsistent database state.
- How to check whether a schedule is serializable?

Checking whether a schedule is serializable

- Swap non-conflicting operations to get serial schedule
- Non-conflicting operations? any two operations that:
 - Operate on different data items.
 - Operate on the same element and both are reads.
- Assume we swap the operations of a schedule S to reach a schedule S' .
 - We say S and S' are conflict-equivalent.
 - If S' is serial then S is serializable.
 - If S is serializable then the transactions in S are isolated.

Executing all transactions in S leads to a consistent database.

Examples on Serializable Schedules

Is this schedule serializable?

T1:	R(A), W(A),	R(B), W(B)
T2:	R(A), W(A)	R(B), W(B)

Examples on Serializable Schedules

Is this schedule serializable?

T1:	R(A), W(A),	R(B), W(B)
T2:	R(A), W(A)	R(B), W(B)

Examples on Serializable Schedules

Is this schedule serializable? **No**

T1:	R(A), W(A),	R(B), W(B)
T2:	R(A),W(A) R(B),W(B)	

Why?

We can't reach on of these serial schedule by swapping operations in the schedule above.

T1:	R(A), W(A), R(B), W(B)
T2:	R(A),W(A) R(B),W(B)

T1:	R(A), W(A), R(B), W(B)
T2:	R(A),W(A) R(B),W(B)

Concurrency Control

- Schedule
- Serial Schedule
- Serializable schedule
 - How to check whether schedule is serializable (using swapping operations)

Can we use the operation-swapping method in practice?

T1:	R(A), W(A),	R(B), W(B)
T2:	R(A), W(A)	R(B), W(B)

Can we use the operation-swapping method
in practice?

T1:	R(A), W(A),	R(B), W(B)
T2:	R(A), W(A)	R(B), W(B)

No,
We can't have the schedule in advance



Strict 2 Phase Locking

- Swapping operations to check schedule serializability is **not practical**.
 - We will need to know schedules in advance (impossible!).
- We need something better.
 - Strict 2 Phase Locking (S2PL)
 - Multi-Version Concurrency Control (MVCC)

Locking: Used in Concurrency Control

- Transaction must get a lock – before it can read or update a data item
- There are two kinds of locks:
shared (S) locks and exclusive (X) locks
- To read a record you MUST get an S lock
To modify or delete a record you MUST get an X lock
- Lock info maintained by a “lock manager”



How Locks Work

- If an object has an **S** lock, new transactions can get **S** locks but not **X** locks.
- If an object has an **X** lock, no other transaction can get any lock (**S** or **X**) on that object.
- If a transaction can't get a lock, it *waits*.

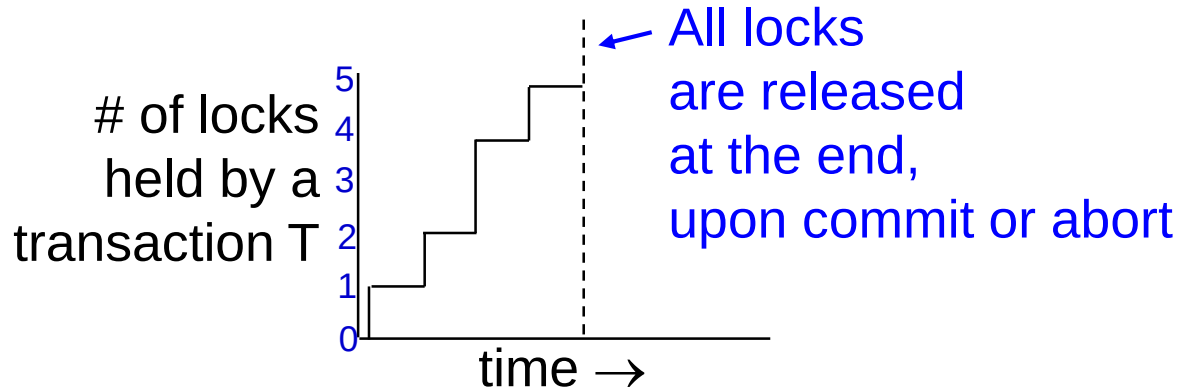
		lock on data item		
lock you want		none	S	X
	S	ok	ok	no
	X	ok	no	no

Lock compatibility matrix

Strict Two Phase Locking Protocol (S2PL)

Strict **2PL** is a way of managing locks during a transaction T

- T gets (S and X) locks gradually, as needed
- T holds all locks until end of transaction (commit/abort)



Try to enforce 2PL on each schedule

T1:	R(A),	W(A)
T2:	R(A),	W(A), R(B)

T1:	R(B),	W(A), W(B)
T2:	R(A),	W(A), R(B)

T1:	R(A),	R(B),	W(B)
T2:	W(A)		
T3:		W(A)	R(B)

Try to enforce 2PL on each schedule

T1:	S(A) R(A),	W(A)	Schedule cannot arise
T2:	S(A) R(A),	X(A) W(A), R(B)	

T1:	S(B) R(B),	X(A) W(A), X(B) W(B)	Rel.	can arise
T2:	S(A) R(A),	X(A) W(A), S(B) R(B)	Rel.	

T1:	S(A) R(A),	R(B),	W(B)	Schedule cannot arise
T2:	X(A)	W(A)		
T3:		W(A)	R(B)	

Text in green indicates successful acquirement of locks and releases. Text in red indicates unsuccessful acquirement of locks.

Strict 2PL guarantees serializability

- Can prove that a Strict 2PL schedule is equivalent to a serial schedule in which each transaction runs instantaneously at the time that it commits

Supporting the ACID Properties of Transactions

- Recovery System* • **Atomicity:** A transaction happens in its entirety or not at all.
- Programmers* • **Consistency:** If the DB starts in a consistent state, (this notion is defined by the user; some of it may be enforced by integrity constraints) and a transaction executes with no other queries active, then the DB ends up in a consistent state.
- Concurrency Control System* • **Isolation:** The transaction is isolated from other transactions. The effect on the DB is (as if) the transaction executed by itself. Other transactions don't see partial updates.
- Recovery System* • **Durability:** If a transaction commits, its changes to the database state persist.

Cost of S2PL

- S2PL provides perfect isolation and therefore perfect consistency.
 - However, there are disadvantages
 - Overhead of locking
 - Less concurrency (a transaction may have to wait to acquire a lock).
 - Means slower application ☹
 - Tradeoff. What do you want?
 - More consistency (more data correctness), but less concurrency (slow application)
 - Or less consistency (less data correctness), but higher concurrency (faster application)
 - Or something in between!
-

Isolation Levels

- Isolation levels
 - READ UNCOMMITTED
 - READ COMMITTED
 - REPEATABLE READ
 - SERIALIZABLE
- There are choices of isolation levels that help you tune the consistency-concurrency tradeoff.

Isolation Levels

- Isolation levels
 - READ UNCOMMITTED
 - READ COMMITTED
 - REPEATABLE READ
 - SERIALIZABLE
- Picking the isolation level in MySQL:
- SET TRANSACTION ISOLATION LEVEL **SERIALIZABLE**;
- Start Transaction
-
- Commit

Isolation Levels

- Default isolation level:
 - READ UNCOMMITTED
 - READ COMMITTED -> Oracle, PostgreSQL, SQL Server
 - REPEATABLE READ -> MySQL
 - SERIALIZABLE -> SQLite

Isolation levels: How they work

- **READ UNCOMMITTED**
 - Does not bother with locks when reading
- **READ COMMITTED**
 - Obtains write locks, releases read locks immediately
- **REPEATABLE READ**
 - Locks only individual objects (not whole table)
- **SERIALIZABLE**
 - Locks sets of objects (tables, pages, index ranges)
- Different isolation levels provide different isolation and consistency levels.

Isolation levels: How they work

- READ UNCOMMITTED
 - Does not bother with locks when reading
- READ COMMITTED
 - Obtains write locks, releases read locks immediately
- REPEATABLE READ
 - Locks only individual objects (not whole table)
- SERIALIZABLE
 - Locks sets of objects (tables, pages, index ranges)
- Serializable and Repeatable Read are equivalent to S2PL.
 - The difference is the granularity (e.g., lock a row or lock a whole table!)

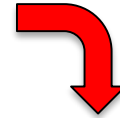
James may argue that he was hired (added to the table) while bonus was added to instructors, but he did not get the bonus

Phantom Data

```
select ID, name  
from instructor  
where salary > 90000;
```



Add a bonus to
all instructors
whose salary >
90000



```
insert into instructor values ('11111', 'James', 'Marketing', 100000);
```

- Two options:
 - **Serializable:** Lock whole table or index range -> **phantom data is not permitted** (e.g., James will not be added while bonus is given to instructors).
 - **Repeatable Read:** Lock only individual objects (not whole table) -> **allows phantom data** (e.g., James will be added while we add bonus to instructors this may create a discrepancy/issues)

Isolation levels: How they work

- **READ UNCOMMITTED**
 - Does not bother with locks when reading
- **READ COMMITTED**
 - Obtains write locks, releases read locks immediately
- **REPEATABLE READ**
 - Locks only individual objects (not whole table)
- **SERIALIZABLE**
 - Locks sets of objects (tables, pages, index ranges)

Isolation levels: How they work

- READ UNCOMMITTED
 - Does not bother with locks when reading
- READ COMMITTED
 - Obtains write locks, releases read locks immediately
- REPEATABLE READ
 - Locks only individual objects (not whole table)
- SERIALIZABLE
 - Locks sets of objects (tables, pages, index ranges)
- Serializable and Repeatable Read are equivalent to S2PL.
 - The difference is the granularity (e.g., lock a row/page or lock a whole table!)

S2PL, release all locks at the end of a transaction

T1:	S(B) R(B),	X(A)W(A), X(B)W(C) Rel.
T2:	S(A)R(A),	X(A) W(C),S(B) R(B) Rel.

What is the disadvantage of keeping locks until the end?

Isolation levels: How they work

- **READ UNCOMMITTED**
 - Does not bother with locks when reading
- **READ COMMITTED**
 - Obtains write locks, releases read locks immediately
- **REPEATABLE READ**
 - Locks only individual objects (not whole table)
- **SERIALIZABLE**
 - Locks sets of objects (tables, pages, index ranges)

READ COMMITTED

releases read locks immediately (does not wait until the end)

T1:	S(A) R(A) Rel(A)	S(A) R(A) Rel(A) ...
T2:	X(A) W(A) Commit.	
T3:	X(A)W(A) Commit	

T1 Releases a read lock on A immediately after the first read

Therefore, T3 changes A even though T1 has not committed yet

Therefore, T1 reads two different values of A

In other words, getting the same value in multiple reads (repeatable reads) is not guaranteed.

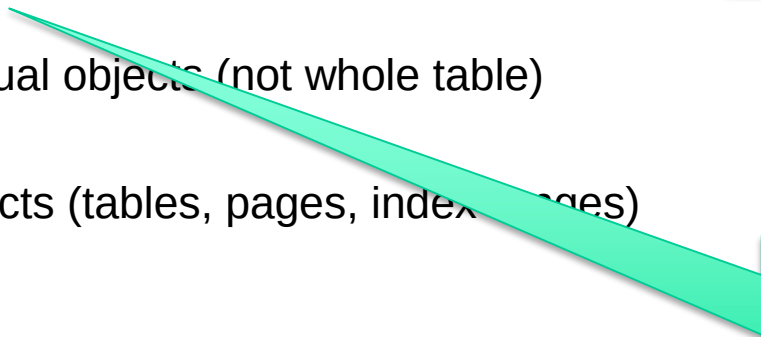
One benefit: T3 did not have to wait for T1 to finish (more concurrency)

Isolation levels: How they work

- **READ UNCOMMITTED**
 - Does not bother with locks when reading
- **READ COMMITTED**
 - Obtains write locks, releases read locks immediately
- **REPEATABLE READ**
 - Locks only individual objects (not whole table)
- **SERIALIZABLE**
 - Locks sets of objects (tables, pages, index pages)



Repeatable Reads
are NOT
guaranteed



Repeatable
Reads are
guaranteed

READ COMMITTED

Obtains write locks, releases read locks immediately

T1:	S(A) R(A) Rel(A)	S(A) R(A) Rel(A) ...
T2: X(A) W(A) Commit.		
T3:	X(A)W(A) Commit	

Why it is called READ COMMITTED?

Reads only committed data (different from read uncommitted in next slide)

Isolation levels: How they work

- **READ UNCOMMITTED**
 - Does not bother with locks when reading
- **READ COMMITTED**
 - Obtains write locks, releases read locks immediately
- **REPEATABLE READ**
 - Locks only individual objects (not whole table)
- **SERIALIZABLE**
 - Locks sets of objects (tables, pages, index ranges)

There is no
S(A) before
the read!!

READ UNCOMMITTED

Does not bother with locks when reading

Called dirty read!

T1:	R(A)	X(A) W(A) Commit.
T2:	X(A) W(A)	W(A) Commit.

T1 reads A without obtaining a shared lock on A

Therefore T1 read uncommitted data (A is updated by T2 but not committed yet)

What if T2 aborts?

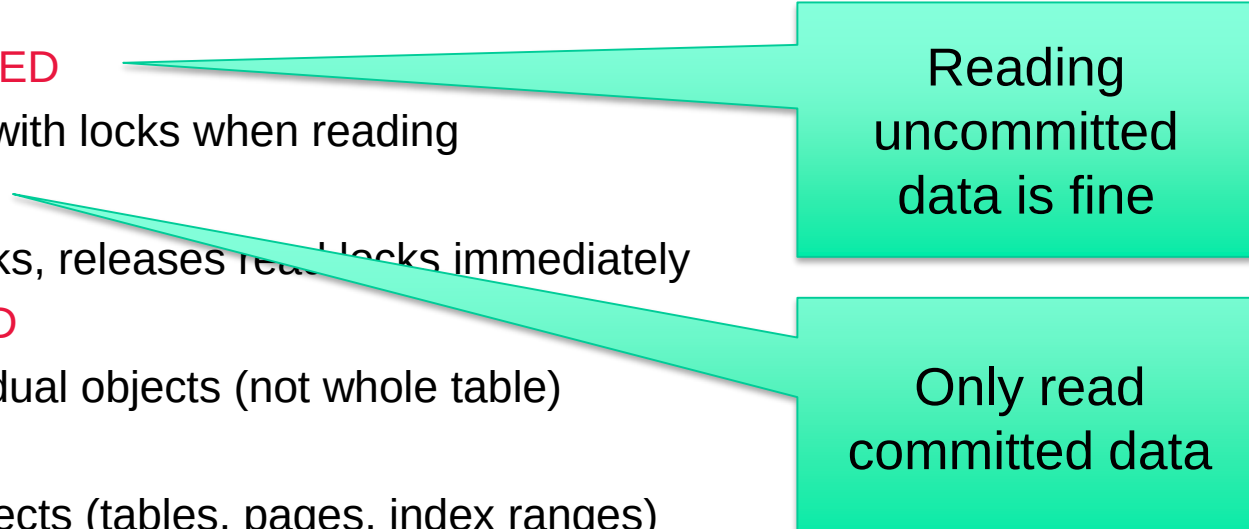
More concurrency than READ COMMITTED

If we used READ COMMITTED, T1 would have to wait for T2 to commit.

Example usage: A transaction collecting **approximate** stats about students (Average GPA, average age, ... etc.). It is OK if the stats are slightly inaccurate!

Isolation levels: How they work

- **READ UNCOMMITTED**
 - Does not bother with locks when reading
- **READ COMMITTED**
 - Obtains write locks, releases read locks immediately
- **REPEATABLE READ**
 - Locks only individual objects (not whole table)
- **SERIALIZABLE**
 - Locks sets of objects (tables, pages, index ranges)



Reading
uncommitted
data is fine

Only read
committed data

Isolation levels (example in MySQL)

Example in MySQL:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
START TRANSACTION;
```

```
SELECT * FROM Student ...  
UPDATE .....  
INSERT .....  
SELECT .....
```

```
COMMIT;
```

The tradeoff between consistency and concurrency

More consistency -> less concurrency

Less consistency -> more concurrency



More consistency = increase data correctness

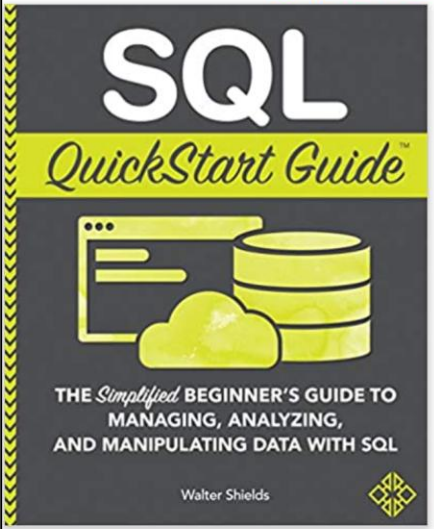
More concurrency = increase availability for system features (faster response time)

The tradeoff between consistency and concurrency

- Applications where more consistency is important
 - Banking, medical data, plant control system
- Applications where more concurrency is important (also less consistency might be tolerated)
 - applications where slight inaccuracy is acceptable but availability is crucial (e.g., social networks!)
 - What is more important, the ability to fetch a Ed Stem question/post more quickly or the accuracy of the number of likes?

When to start the transaction

Look inside ↓



ISBN-13: 978-1945051753
ISBN-10: 1945051752

Kindle 	Audiobook	Hardcover	Paperback	Other Sellers
\$9.99	\$0.00	\$28.22 - \$28.24	\$23.74	See all 5 versions

Buy new:

In Stock.

Ships from and sold by Amazon.com.

FREE delivery: **Sunday, Nov 1** [Details](#)

\$23.74

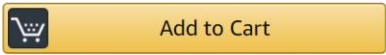
List Price: ~~\$24.99~~
Save: \$1.25 (5%)

8 new from \$23.74



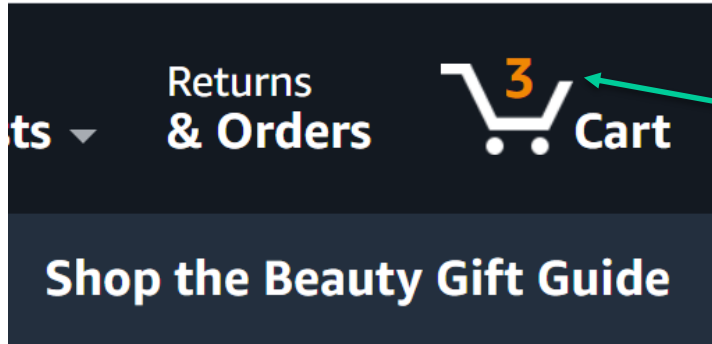
 Deliver to Hisham - West Lafayette 47906

Qty: 1 ▼

 Add to Cart

 Buy Now

Should we start the transaction when the user clicks on Add to Cart or when the user checks out?



What if the user keeps shopping for a long time

That would be a long transaction

Possibly, locking many data items.



Place your order

Better alternative:

start the transaction, process the data, and commit/abort when the user reaches the final step

Any problems with this approach?

The data the user has seen is not locked and can change.

For example, the number of book copies was 2 when the user started shopping, but became zero when the user tried to place the order.

ATM Scenario



Scenario:

Customer: inserts card

System:

Start Transaction;

Read Balance;

Display Balance;

Wait for Customer;

Customer: withdraw money

System: update balance;

ATM Scenario



While the transaction is running, the balance is locked (assuming strong isolation level).

This is a family account. The husband is using the ATM while his wife is in the grocery store trying to check out by doing a debit purchase with no success (debit purchase transaction cannot update account balance).

Another idea

Steps

Step 1

Step 2

Step 3

Step 4

Step 5

Payment

Your chosen test is not confirmed until you have completed your booking.



Your test selection is being held for the next 30 minutes. You should complete registration in this time to avoid losing this space.

Your chosen test will be held for: **29:39**

IELTS Academic

Listening, Reading and Writing

Supporting the ACID Properties of Transactions

- Recovery System* • **Atomicity:** A transaction happens in its entirety or not at all.
- Programmers* • **Consistency:** If the DB starts in a consistent state, (this notion is defined by the user; some of it may be enforced by integrity constraints) and a transaction executes with no other queries active, then the DB ends up in a consistent state.
- Concurrency Control System* • **Isolation:** The transaction is isolated from other transactions. The effect on the DB is (as if) the transaction executed by itself. Other transactions don't see partial updates.
- Recovery System* • **Durability:** If a transaction commits, its changes to the database state persist.

A word about recovery/durability

- The recovery system ensures atomicity and durability of transactions in the event of a system crash or power failure.
 - Using a log of all update operations.
- Recovery system does not protect from disk failure.
 - You need to back up your database periodically (e.g., using mysqldump command in MySQL).
 - Can use replicas as well (RAID system or multiple replicas in the cloud)
 - Each update is propagated to all replicas (e.g., three replicas always have the same up-to-date database)
 - Can slow down response time especially if replicas are in different regions (e.g., one in US east coast, one in Singapore, and one in Ireland)

Summary

- Use transactions: Begin transaction, some reads and updates, commit/roll back.
 - Recovery and concurrency control systems will ensure atomicity, durability, and isolation for your transactions.
 - And hence consistency of your data.
- Choose your isolation level based on how accurate the data need to be and how much concurrency you would like to have.
- Be aware of long transactions that reduce concurrency (by locking data for a long time)